

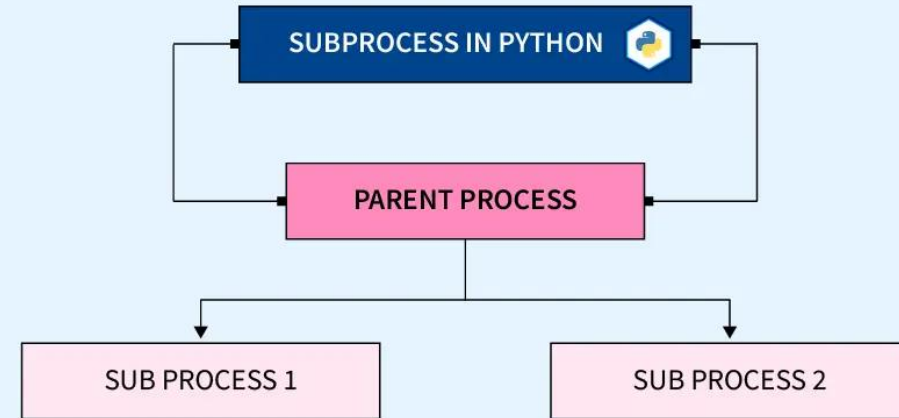


Welcome & Module Induction

- **What we learned previously:** In Module 5, we learned how to engineer files—moving, organizing, and reading the metadata of thousands of files using Python's `os` and `pathlib` modules.
- **What we are learning today:** We are moving from controlling *files* to controlling *other programs*. We will learn how to use Python to act as a "remote control" for the terminal and other software using the `subprocess` module and System Exit Codes.
- **Why this will be helpful:** You will not write every piece of software you use in your career. Sometimes you need to use FFmpeg to process video, Git to sync code, or a specialized C++ binary to run a computer vision model. Instead of opening these apps manually, you can write a Python script that automatically opens them, feeds them data, reads their output, and handles their errors.

The subprocess Module

- **The Concept:** Think of `subprocess` as a virtual human sitting at a terminal. It types commands, hits "Enter," and waits for the program to finish.
- **Why not `os.system()`?** Older tutorials use `os.system()`, but it is outdated and insecure. `subprocess.run()` is the modern, industrial standard because it gives you absolute control over the input, output, and security of the program you are launching.



Running Shell Commands & Capturing Output

- **The Problem:** When you run a terminal command, the output usually prints directly to the screen. But what if your Python script needs to *read* that output to make a decision?
- **The Solution:** We use `capture_output = True`. This catches the text before it hits the screen and stores it in a Python variable (`result.stdout`).
- **Example:** You can run `git status`, capture the output, and if the text contains the word "*modified*", your Python script can automatically trigger a backup.



```
1 import sys
2
3 def main():
4     print("Executing main function")
5     sys.exit("Program terminated using sys.exit()")
6
7 if __name__ == "__main__":
8     main()
```



System Exit Codes (How Programs Talk)

- The Hidden Language:** When a program finishes, it sends a hidden number back to the Operating System. This is called the Exit Code (or Return Code).
- The Universal Rule:** * 0 = Absolute Success.
- 1 (or any non-zero number) = Error/Failure.
- Why it matters:** If your Python script triggers a 5-hour video rendering process in FFmpeg, it needs to know if it actually worked. By checking the Exit Code, your script knows whether to proceed to the next step or send an emergency error log.